

Spółeczna Akademia Nauk

Instytut Technologii Informatycznych

Kierunek studiów:

INFORMATYKA

Przedmiot:

Systemy Szkieletowe

**Debilingo – aplikacja webowa wspierająca naukę języków
obcych z elementami grywalizacji**

Maksym Mokriakov

Nr albumu: 121261

Grupa: 2

Prowadzący:

mgr inż. Krystian Gumiński

Łódź 2026

Spis treści

1	Wstęp	2
2	Cel projektu	3
3	Przegląd istniejących rozwiązań	4
3.1	Duolingo	4
3.2	Quizlet	4
3.3	Anki	4
4	Opis architektury	5
5	Opis implementacji	6
5.1	Frontend	6
5.2	Backend	7
5.3	Baza Danych	7
5.4	Docker	7
5.5	Testy	8
6	Podsumowanie i wnioski	9
	Bibliografia	12

1 Wstęp

Nauka języków obcych jest procesem długotrwałym, który wymaga regularności i dużego zaangażowania. W dobie powszechnego dostępu do urządzeń mobilnych, aplikacje wspomagające naukę stały się niezwykle popularne, jednak wiele z nich traci użytkowników z powodu szybkiego spadku motywacji po pierwszych dniach korzystania.

Niniejszy projekt, noszący nazwę "Debilingo", ma na celu rozwiązanie tego problemu. Aplikacja łączy w sobie funkcjonalność osobistego słownika do nauki obcych słówek z silnymi mechanizmami grywalizacji (gamifikacji). Użytkownicy są nagradzani za regularność punktami doświadczenia (XP), rozwijają swój wirtualny poziom oraz budują "Daily streak"(codzienną serię). W raporcie przedstawiono architekturę rozwiązania, zastosowane technologie oraz kluczowe aspekty implementacyjne.

2 Cel projektu

Głównym celem projektu jest stworzenie nowoczesnej i responsywnej aplikacji webowej, zoptymalizowanej pod kątem urządzeń mobilnych, która umożliwi użytkownikom efektywną naukę słownictwa poprzez samodzielnie tworzone listy słów połączone z elementami motywującymi.

Wymagania funkcjonalne obejmują:

- Rejestrację, logowanie oraz zarządzanie profilem użytkownika.
- Tworzenie własnych, spersonalizowanych słowników językowych.
- Dodawanie par słówek do wybranych słowników.
- Śledzenie postępów użytkownika (system poziomów, naliczanie punktów XP, codzienne serie).

Wymagania niefunkcjonalne to między innymi wysoka wydajność, niezawodność, zastosowanie architektury "Mobile-first" oraz pełne pokrycie kluczowych funkcjonalności testami automatycznymi. Skalowalność systemu zapewniono dzięki chmurze obliczeniowej i konteneryzacji.

3 Przegląd istniejących rozwiązań

Na rynku istnieje wiele aplikacji wspierających naukę języków, z których każda kładzie nacisk na inne aspekty edukacji. Analiza tych systemów pozwoliła na zidentyfikowanie zarówno dobrych praktyk, jak i obszarów wymagających poprawy.

3.1 Duolingo

Duolingo jest obecnie jedną z najpopularniejszych aplikacji do nauki języków na świecie. Jej ogromną zaletą jest niezwykle rozbudowany system grywalizacji, który silnie motywuje do codziennych ćwiczeń. Wadą dla zaawansowanych użytkowników jest jednak z góry narzucony program nauczania i brak możliwości wprowadzania własnego słownictwa z określonej branży (np. IT).

3.2 Quizlet

Quizlet to platforma oparta na wirtualnych fiszkach (flashcards), pozwalająca na tworzenie własnych zestawów słówek. Jest to bardzo elastyczne narzędzie, często wykorzystywane w szkołach i na uczelniach. Niestety, w darmowej wersji brakuje zaawansowanych mechanizmów podtrzymujących zaangażowanie i długotrwałą motywację (elementów gry), przez co proces nauki po pewnym czasie może wydawać się nużący.

3.3 Anki

Anki to potężne oprogramowanie wykorzystujące algorytm Spaced Repetition (rozłożonych w czasie powtórek). Pozwala na naukę czegokolwiek, od słówek po medycynę. Choć jest niesamowicie skuteczne, jego interfejs jest archaiczny i skomplikowany dla nowych użytkowników, a brak jakiegokolwiek grywalizacji sprawia, że wymaga żelaznej dyscypliny.

Debilingo stara się połączyć najlepsze cechy wyżej wymienionych systemów: elastyczność w dodawaniu własnych słówek (Quizlet) oraz wciągające mechanizmy motywacyjne i łatwy w obsłudze interfejs (Duolingo). Na rysunku 1a przedstawiono poglądowy widok aplikacji realizującej te założenia.

4 Opis architektury

System został zaprojektowany z wykorzystaniem architektury rozproszonej typu klient-serwer, opierając się o nowoczesne rozwiązania chmurowe. Tabela 1 prezentuje podział na główne warstwy.

Tabela 1: Główne komponenty architektury systemu

Lp.	Komponent	Technologia i rola
1	Frontend (Aplikacja kliencka)	Vercel. Odpowiada za warstwę prezentacji (UI/UX). Oparta o React.
2	Backend (API)	Google Cloud Run. Realizuje logikę biznesową (Node.js, Express) z zachowaniem pełnej skalowalności.
3	Baza danych	Supabase (PostgreSQL). Platforma BaaS przechowująca dane użytkowników i słowniki.

Aplikacja frontendowa komunikuje się z serwerem backendowym za pośrednictwem żądań HTTP (REST API). Część zadań, takich jak natychmiastowe autoryzowanie sesji, wspierana jest również bezpośrednio przez klienta biblioteki Supabase w przeglądarce. Serwer backendowy jest całkowicie bezstanowy, co umożliwia jego bezpieczną konteneryzację w usłudze Google Cloud Run.

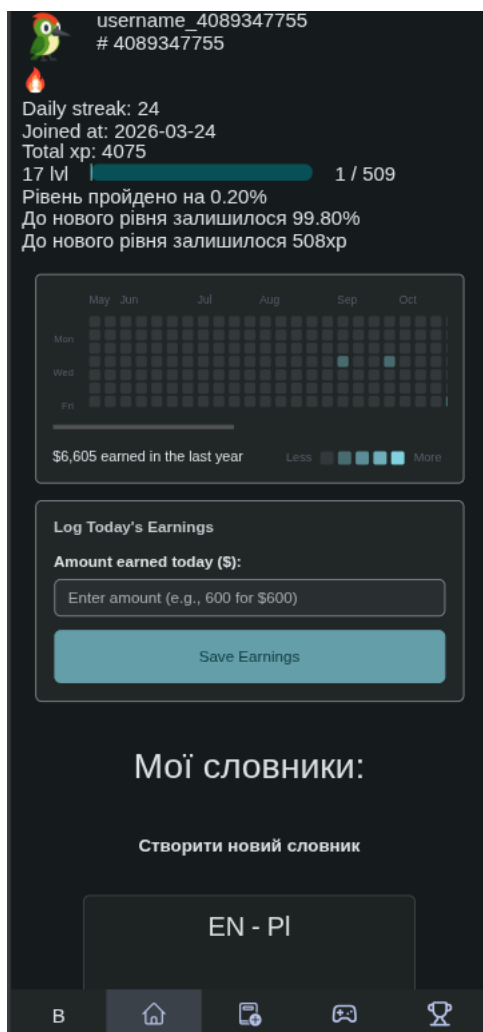
5 Opis implementacji

Implementacja systemu została podzielona na odrębne moduły zarządzane w ramach struktury monorepozytorium (oddzielne katalogi web-app oraz backend).

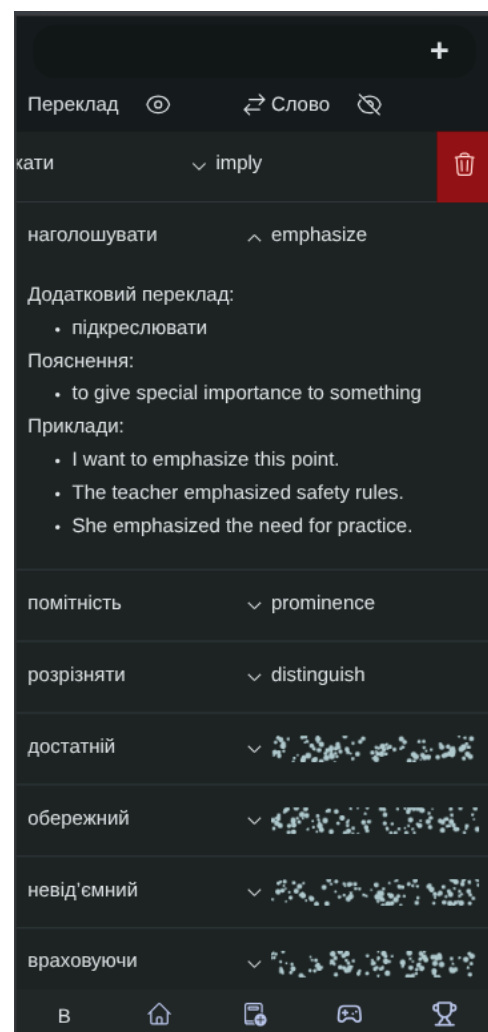
5.1 Frontend

Frontend został zaimplementowany jako aplikacja Single Page Application (SPA) przy użyciu biblioteki React [1] (w wersji 19) oraz języka TypeScript [2]. Jako narzędzie budujące wykorzystano Vite [3], co znacząco przyspieszyło proces developmentu. Architektura kodu po stronie klienta oparta jest na nowoczesnej metodologii Feature-Sliced Design (FSD) [4], co widać w strukturze katalogów (app, entities, features, pages, shared, widgets). Zarządzanie stanem globalnym, w tym śledzenie punktów XP oraz trwającej serii użytkownika, powierzono bibliotece Zustand [5].

Interfejs został zaprojektowany zgodnie z podejściem Mobile-first.



(a) Strona główna



(b) Словник

Rysunek 1: Poglądowy interfejs aplikacji mobilnej

5.2 Backend

Warstwa serwerowa została napisana z wykorzystaniem środowiska uruchomieniowego Node.js oraz frameworka Express.js. Całość zaimplementowano w języku TypeScript dla zachowania typowania (Type-Safety) na każdym etapie. Głównym zadaniem backendu jest obsługa zaawansowanej logiki biznesowej, bezpieczna wymiana danych oraz zarządzanie słownikami użytkowników.

5.3 Baza Danych

Projekt wykorzystuje usługę Supabase [6], będącą otwartoźródłową alternatywą dla Firebase, pod spodem opierającą się na relacyjnej bazie danych PostgreSQL. Dzięki zastosowaniu mechanizmów Row Level Security (RLS) wbudowanych w Supabase, dane są całkowicie bezpieczne na poziomie bazy – każdy użytkownik ma wgląd i możliwość modyfikacji wyłącznie własnych słowników i postępów.

5.4 Docker

W celu łatwego i bezawaryjnego przenoszenia środowiska aplikacyjnego, backend został w pełni skonteneryzowany z użyciem technologii Docker [7]. Zbudowany obraz kontenera zawierający Node.js, zainstalowane zależności (`node_modules`) i zbudowane pliki źródłowe, pozwala na uruchomienie aplikacji w dowolnym środowisku produkcyjnym – ostatecznie wybrano elastyczną usługę Google Cloud Run [8], która skaluje ilość instancji w zależności od obciążenia.

```
backend > Dockerfile
1 FROM node:22-alpine
2
3 WORKDIR /app
4
5 COPY package*.json ./
6 COPY tsconfig.json ./
7
8 RUN npm install
9
10 COPY src ./src
11
12 RUN npm run build
13
14 EXPOSE 8080
15
16 CMD ["node", "dist/server.js"]
17
```

Rysunek 2: Dockerfile

5.5 Testy

Biorąc pod uwagę jakość wytworzonego oprogramowania, wprowadzono rygorystyczne procedury testowe. Do testów jednostkowych (Unit tests) na frontendzie użyto nowoczesnego i niezwykle szybkiego narzędzia Vitest, które współpracuje z systemem budowania Vite. Dodatkowo, w projekcie zaimplementowano testy end-to-end (E2E) korzystając z frameworka Playwright, które symulują realne zachowanie użytkownika przechodzącego przez ekrany aplikacji. Kod sprawdzany jest również linterem (ESLint) oraz formaterem (Prettier).

6 Podsumowanie i wnioski

W ramach zrealizowanego projektu stworzono w pełni funkcjonalną i nowoczesną aplikację wspomagającą naukę języków obcych (Debilingo). Udało się pomyślnie zintegrować mechanizmy grywalizacji z narzędziem typu "flashcards", co znacząco zwiększa motywację do codziennej nauki. Zastosowanie technologii takich jak React 19, TypeScript oraz Supabase znacznie skróciło czas tworzenia oprogramowania i pozwoliło skupić się na dostarczeniu intuicyjnego interfejsu (UX).

Największym wyzwaniem w projekcie okazało się prawidłowe zaprojektowanie relacji w bazie danych PostgreSQL oraz obsługa stanu globalnego na frontendzie, co zostało skutecznie rozwiązane dzięki elastyczności biblioteki Zustand. System jest otwarty na rozbudowę. W przyszłości planowane jest dodanie m.in. systemu znajomych, globalnych rankingów użytkowników oraz generowania przykładowych zdań dla nowych słówek przy pomocy sztucznej inteligencji.

Spis tabel

1	Główne komponenty architektury systemu	5
---	--	---

Spis rysunków

1	Poglądowy interfejs aplikacji mobilnej	6
2	Dockerfile	7

Bibliografia

- [1] Meta Platforms, Inc. *Using TypeScript with React - React Documentation*. URL: <https://react.dev/learn/typescript> (dostęp 22.05.2026).
- [2] Microsoft Corporation. *TypeScript for JavaScript Programmers*. URL: <https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html> (dostęp 22.05.2026).
- [3] Vite Core Team. *Features - Vite Documentation*. URL: <https://vitejs.dev/guide/features.html> (dostęp 22.05.2026).
- [4] FSD Community. *Overview - Feature-Sliced Design Documentation*. URL: <https://feature-sliced.design/docs/get-started/overview> (dostęp 22.05.2026).
- [5] Poimandres. *Zustand - Bear necessities for state management in React*. URL: <https://docs.pmnd.rs/zustand/getting-started/introduction> (dostęp 22.05.2026).
- [6] Supabase Inc. *Row Level Security - Supabase Documentation*. URL: <https://supabase.com/docs/guides/auth/row-level-security> (dostęp 22.05.2026).
- [7] Docker Inc. *Containerize a Node.js application - Docker Documentation*. URL: <https://docs.docker.com/language/nodejs/containerize/> (dostęp 22.05.2026).
- [8] Google LLC. *What is Cloud Run? - Google Cloud Architecture Center*. URL: <https://cloud.google.com/run/docs/overview/what-is-cloud-run> (dostęp 22.05.2026).